

# PsumQuant: In-line Post-training Partial Sum Quantizer for Energy Efficient NPU Inference

Sangwoo Hwang<sup>1</sup> Yeeun Hong<sup>2</sup> Jaeha Kung<sup>2</sup>

## Abstract

The rapid growth of deep neural networks (DNNs) has intensified the demand for efficient hardware acceleration under quantization. While prior research has successfully reduced weight and activation precision, partial sums generated during accumulation often retain high precision, resulting in significant energy overhead. In this work, we analyze psum distributions in tiled architectures and reveal that within-tile outliers are input-dependent. We propose PsumQuant, a post-training, input-aware quantization that predicts psum scales on-the-fly. By leveraging the crest factor of input activations, our learnable scale predictor effectively bounds the psum bit-width while handling the extreme outliers in DNNs. Experimental results on a  $128 \times 128$  systolic array demonstrate that PsumQuant compresses psum precision down to 8-bit within only a 1% accuracy drop on ResNet-18 and a marginal 0.04 perplexity increase on Llama-3.1. Furthermore, bit-width reduction with PsumQuant results in a 45% reduction in total energy with minimal accuracy loss, demonstrating that PsumQuant provides a highly efficient solution for actual NPU architectures.

## 1. Introduction

Recent deep neural networks (DNNs) have fundamentally changed human-machine interaction by moving from a command-based to a conversational paradigm. However, to exploit diverse multimodal data in DNNs, significant computational resources and memory are required. Therefore, modern accelerators are carefully designed to maximize throughput and energy efficiency under tight memory and bandwidth constraints (Google Cloud, 2026; Groq, 2026;

<sup>1</sup>Semiconductor Technology Research Institute, Korea University  
<sup>2</sup>School of Electrical Engineering, Korea University. Correspondence to: Jaeha Kung <jhkung@korea.ac.kr>.

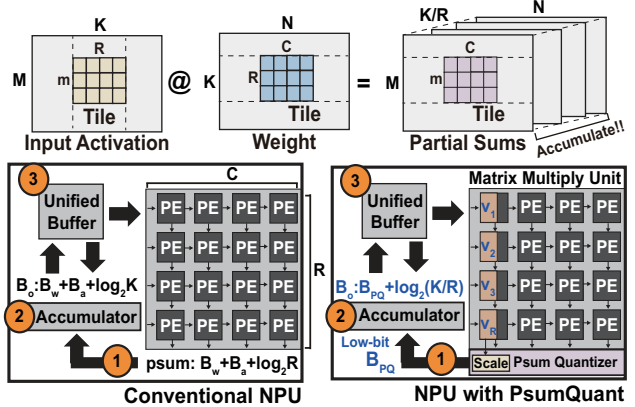


Figure 1. Overview of psum accumulation in tiled GEMM and the motivation for psum quantization. Top: Tiled matrix multiplication produces partial sums that must be reduced across tiles. Bottom-left: In a conventional NPU, partial sums grow in bitwidth during accumulation, leading to increased buffer and accumulator cost. Bottom-right: The proposed NPU integrates a *psum quantizer* to minimize psum precision during accumulation, alleviating buffer pressure and data-movement overhead.

Cerebras Systems, 2026). As a result, 2D array-based architectures, including systolic arrays (Jouppi et al., 2017), have become a widely adopted standard for high-performance DNN acceleration.

Figure 1 illustrates a tiled matrix multiplication executed on an  $R \times C$  systolic array. Due to limited on-chip storage, input activations and weights are partitioned into tiles (i.e., *loop tiling*) that are sequentially streamed into the array, while each processing element performs multiply-accumulate operations on the incoming operands. The results of these tiled computations are accumulated as intermediate partial sums (psums) before producing the final output. Tiling is a crucial step in the compilation of DNN workloads prior to running them on systolic arrays (or other 2D spatial accelerators), ensuring high resource utilization under given memory constraints.

By tiling, the intermediate ‘psum’ tiles in the output buffer should be accumulated to prevent memory overflow and preserve numerical correctness. The continuous traffic of psum tiles between the compute array and the global buffer frequently becomes a dominant factor in on-chip bandwidth

and dynamic energy consumption. Furthermore, psum often requires substantially higher bitwidth than either weights or activations to ensure a wide dynamic range and finer precision. For example, each psum tile aggregates  $R$  products between  $B_w$ -bit weights and  $B_a$ -bit activations, making the required precision become ‘ $\log_2 R + B_w + B_a$  bits’. Thus, even if activation and weight are quantized to low precision, the psums remain significantly larger in bit-width. **(Motivation)** Nevertheless, quantization of psum tiles remains overlooked, unlike weight and activation tensors (Jacob et al., 2018; Han et al., 2016; Dettmers et al., 2023).

Our approach reduces psum precision to  $B_{PQ}$  by dynamically scaling psum using an input-dependent factor  $\mathbf{v} \in \mathbb{R}^R$ . Moving beyond static scaling, we introduce a learnable projection vector  $\mathbf{v}$  that aligns the quantization scale with input variations, optimized to minimize local quantization error. Learning  $\mathbf{v}$  is further guided by the crest of psum distributions, allowing the quantizer to focus on outliers more. As a result, psum bit-width is effectively bounded (7~8-bit) during tiled accumulation, significantly reducing ❶ psum traffic, ❷ accumulator bit-width, and ❸ buffer storage requirements (Fig. 1) while preserving numerical fidelity. The main contributions are summarized below:

- **NPU-friendly Psum Quantizer:** We propose *PsumQuant*, an input-aware scale predictor for psum quantization that integrates into existing systolic arrays. By enabling 8-bit psum tiles on a  $128 \times 128$  systolic array, we can achieve a **45% energy reduction**.
- **Outlier-aware Scale Calibration:** We introduce a lightweight, layer-wise scale calibration that mitigates outlier-induced errors via crest factor ( $\kappa_{nz}$ )-aware modulation. Our approach achieves **minimal accuracy loss** at 7-/8-bit ( $\sim 1\%$ ) across diverse architectures without requiring weight retraining.
- **Hardware-Software Co-Design:** We consider the actual hardware implementation when designing the tile-based scale calibration method for psums. Extensive analysis on various systolic array sizes is performed.

## 2. Preliminaries

### 2.1. Tiled Matrix Multiplication on Systolic Arrays

The rapid scaling of modern deep learning models has made matrix multiplication the dominant computational kernel in training and inference. To support efficient computations, neural processing units (NPUs), such as Google’s TPU (Google Cloud, 2026), incorporate specialized matrix-multiply units (e.g., systolic arrays) that deliver massive spatial parallelism and high throughput. However, these accelerators operate under on-chip memory constraints and thus cannot process large matrix multiplications in a single monolithic systolic array. Therefore, it is essential to par-

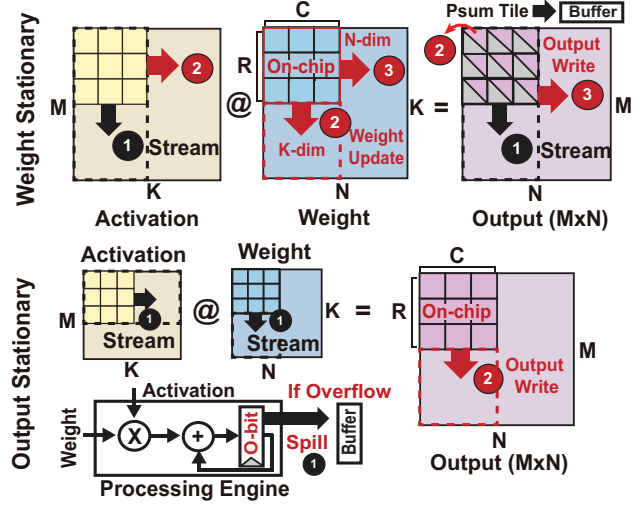


Figure 2. Comparison of weight-stationary (WS) and output-stationary (OS) dataflows under tiled execution. In WS, a weight tile is kept on-chip and reused while activations are streamed in, requiring accumulation of partial sums before writing the final output. In OS, outputs remain local to each PE, but limited register capacity can cause psums to spill into a global buffer.

tition (defined as ‘tiling’) large matrix multiplications into smaller ones that a single systolic array can handle. Fig. 2 illustrates the standard GEMM for computing  $\mathbf{O} \in \mathbb{R}^{M \times N}$  from an input activation matrix  $\mathbf{A} \in \mathbb{R}^{M \times K}$  and a weight matrix  $\mathbf{W} \in \mathbb{R}^{K \times N}$ . We assume an  $R \times C$  systolic array, where the activation, weight, or output tensor cannot be entirely stored in the 2D array. Thus, the computation needs to be tiled according to the amount of data the array can process at once. Tiling strategies and their resulting efficiency depend on the dataflow of the systolic array, e.g., weight-/output-stationary.

#### 2.1.1. WEIGHT-STATIONARY EXECUTION

The weight-stationary (WS) dataflow (Jouppi et al., 2017) is a common architecture for systolic arrays in which weights are locally reused to compute with many input tiles. The WS dataflow keeps a weight tile fixed within a 2D array while streaming activations into the array, so that psums are accumulated and forwarded to the next processing engines (PEs). Generally, to maximize the reuse of weights already stored in the array, input activations are streamed along the  $M$  dimension (❶), producing an  $M \times C$  psum tile using the weights currently resident in the array (Fig. 2(top)). Since the computation is partitioned along the  $K$  dimension, the psum tiles need to be stored in a global output buffer. After the completion of  $M$ -dimension stream, the systolic array updates the weight tile along the reduction dimension  $K$  (❷), and the corresponding  $M$ -dimension streaming (❶) is repeated. This process continues until all  $K \times C$  weight tiles have been used, at which point the generated psum tiles are accumulated to produce the final output (❸). While the

exact scheduling of the WS execution depends on hardware constraints, WS fundamentally minimizes weight traffic but incurs repeated activation traffic and necessitates sufficient buffering for storing intermediate psum tiles.

### 2.1.2. OUTPUT-STATIONARY EXECUTION

In contrast, output-stationary (OS) dataflow (Du et al., 2015) assigns a distinct output element to each PE in an  $R \times C$  systolic array. Activations and weights are streamed along the  $K$  dimension (❶) while each PE accumulates psums for its assigned output in local registers. Once the  $K$ -dimension stream is complete, the PEs are reassigned to a new set of output elements, while the generated output tile is written to the buffer (❷), and the process is repeated. However, when PE register capacity or adder bit-width is limited, these psums cannot be fully retained within the PE, necessitating writes to a dedicated output buffer (*psum spill*). As a result, regardless of whether WS or OS dataflow is employed, buffering of psum tiles is essential and the resulting *data traffic between the compute array and the output buffer introduces large energy overhead*.

### 2.1.3. HARDWARE ANALYSIS FRAMEWORK

To measure the energy cost due to the high precision required for psum tile accumulations, we used an open-source Timeloop-Accelergy simulation framework (Parashar et al., 2019), using the well-known WS architecture (Jouppi et al., 2017) as a baseline. This architecture includes a  $128 \times 128$  systolic array with a three-level memory hierarchy, i.e., DRAM, global buffer, local scratchpad, where the array is modeled in 65nm technology. As shown in Fig. 3, *reducing the psum bit-width from the theoretically needed 24-bit baseline (for  $R = 128$  with 8-bit  $A$  and  $W$ ) to 8-bit results in a promising 48% improvement* in energy efficiency (TOPS/W). The energy breakdown analysis shows that while the energy consumption for input and weight remains constant, the energy cost associated with psum storage and its movement increases greatly with the bit-width. Specifically, at 32-bit precision, handling psums becomes the main source of the total energy consumption.

## 2.2. Quantization Methods

For DNNs, quantization replaces high-precision tensors with low-precision integer representations to reduce memory footprint and improve computational efficiency. Given an FP32 tensor  $\mathbf{X}$ , a uniform quantizer maps each element to an integer tensor  $\mathbf{X}_q$  with bit-width  $b$  using a scale factor  $s \in \mathbb{R}^+$ :

$$\mathbf{X}_q = \text{clamp}\left(\left\lfloor \frac{\mathbf{X}}{s} \right\rfloor, -2^{b-1}, 2^{b-1} - 1\right), \quad (1)$$

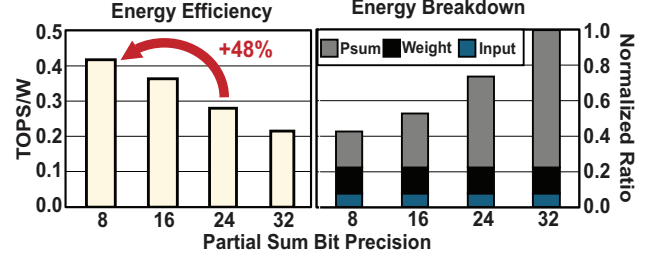


Figure 3. Impact of psum bit-widths on energy-efficiency using a  $128 \times 128$  WS systolic array on GPT2 (Radford et al., 2019). The energy efficiency (left; TOPS/W) is reported for the array running at 1GHz, modeled in 65nm CMOS technology. The energy breakdown (right) shows that psum storage and its movement increasingly dominate total energy as bit-width increases (8 bits for both activations and weights).

where the de-quantized reconstruction is  $\hat{\mathbf{X}} = s \cdot \mathbf{X}_q$ . Thus, the central design choice among quantization methods lies in determining the scale  $s$ . Static quantization determines  $s$  offline, typically via min-max calibration or a distribution-aware method. It incurs no runtime overhead but suffers from a distributional mismatch when activations contain heavy outliers. For example, (Choi et al., 2018; Esser et al., 2020) optimize quantization parameters during training, e.g., clipping thresholds or step sizes. On the other hand, dynamic quantization computes  $s$  at runtime using statistics of the current input (e.g.,  $s = \max |\mathbf{X}| / (2^{b-1} - 1)$ ). While this improves robustness, it requires real-time monitoring of the tensor before quantization can proceed, which introduces additional buffering, latency, and area overheads.

Along the line, a wide range of quantization techniques have been explored across different models, from CNNs to large language models (LLMs), to reduce precision while preserving accuracy. Prior work has investigated post-training quantization (PTQ) (Nagel et al., 2020; Li et al., 2021) and quantization-aware training (QAT) (Jacob et al., 2018; Dong et al., 2019) for both weights and activations with clipping, rescaling, and distribution reshaping. In parallel, binary neural networks (BNNs) (Courbariaux et al., 2016; Rastegari et al., 2016) and spiking neural networks (SNNs) (Tavanaei et al., 2019; Hwang et al., 2024) have been proposed to reduce precision extremely. Challenged by outliers in LLMs, recent methods (Xiao et al., 2023; Ashkboos et al., 2024) improve the quantizability via tensor transformations.

However, prior works have primarily targeted weights and activations because they directly determine the memory footprint of model parameters and the bit-width of multiply operations. However, even when weights and activations are aggressively quantized, psums often remain at a much higher bit-width. To address this gap, recent studies have attempted to reduce the precision of psum computations. A2Q (Colbert et al., 2023) focuses on minimizing the PE-internal accumulator width by constraining weight norms during training to avoid overflow. While recent ap-

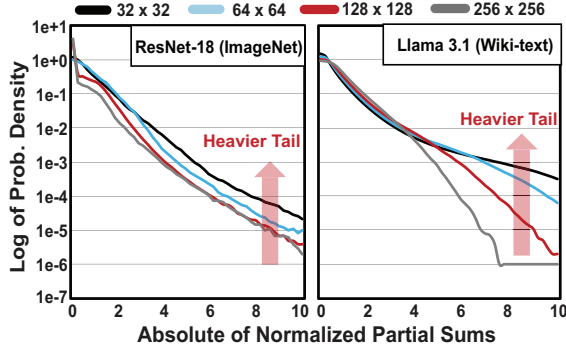


Figure 4. Distribution analysis of normalized psums across different tile sizes. A clear trend is observed where smaller tiles (black line) exhibit significantly heavier tails compared to larger tiles (grey line). Specifically, at  $> 6\sigma$ , the density of small tiles remains orders of magnitude higher than that of large tiles.

proaches for computing-in-memory (CIM) (Bai et al., 2023; Kim et al., 2022; 2025; Saxena et al., 2022) also tackle partial sum quantization, their primary focus is on reducing the bit precision of analog-to-digital converters (ADCs) to minimize area and power consumption within the computing array. However, when bit-serial ADC outputs are shift-added and accumulated in the digital domain, the resulting digital psums inevitably experience bit-growth. This creates a secondary bottleneck in digital data movement and buffer storage outside the computing array. Therefore, addressing this secondary bottleneck requires a dedicated digital quantization stage immediately following the accumulators, indicating that such an approach is fundamentally orthogonal and highly complementary to existing methods. For the same objective, APSQ (Tan et al., 2025) aims to reduce buffer storage overhead by quantizing intermediate psums using a recursive LSQ-based QAT (Esser et al., 2020), which necessitates reconfigurable error-management hardware. However, these approaches fundamentally rely on quantization-aware training, which is expensive, especially for LLMs. In contrast, we focus on an NPU-friendly framework that directly quantizes the psums generated from standard systolic array before they are stored in the buffer. By using a local calibration rather than global retraining, our approach can adapt to various hardware configurations cheaply (*Design Choice #1*).

### 3. Characteristics of Tiled Partial Sums

#### 3.1. Statistical Behavior of Partial Sums

Understanding the feature of psums at the hardware-oriented tile granularity is a prerequisite for achieving low-bit quantization with negligible accuracy loss. While the global distribution provides an overall trend, it does not capture the statistical fluctuations within local tiles where the actual computation occurs. To investigate the practical chal-

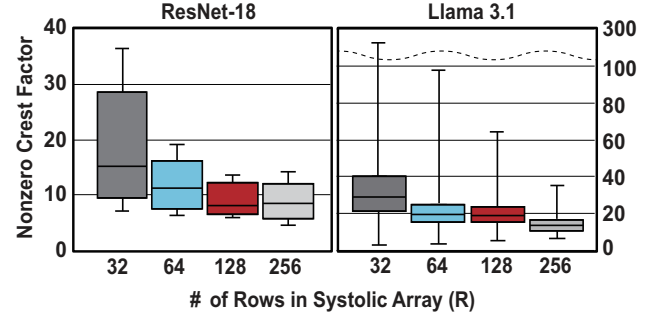


Figure 5. Box plot of the nonzero crest factor  $\kappa_{nz}$  at various tiling granularities which depend on the size of hardware (= systolic array), i.e.,  $32 \times 32$ ,  $64 \times 64$ ,  $128 \times 128$ , and  $256 \times 256$ .

lenges inherent in tile-based quantization, we performed an in-depth analysis of psum distributions across various tile sizes for two representative benchmarks, i.e., ResNet-18 and Llama-3.1 (Dubey et al., 2024) (Fig. 4).

Fig. 4 shows the probability density (in log-scale) of normalized absolute partial sums at various tile sizes, where standardization is performed using the running mean and variance per tile. Large tiles ( $256 \times 256$ ) exhibit a rapid decay in density between the  $4\sigma \sim 6\sigma$  region. In contrast, small tiles ( $32 \times 32$ ) maintain significant density in the same region, forming a distinct heavy tail. In large tiles, the stronger averaging causes the distribution to converge towards a Gaussian shape, effectively diluting outliers. Conversely, small tiles suffer from insufficient accumulation, remaining in a “pre-converged” state where the extreme psums are not smoothed out. Since the outputs from each tile are quantized individually, the quantization error of each tile accumulates additively during the final aggregation, becoming the dominant source of accuracy degradation.

#### 3.2. Nonzero Crest Factor of Tiled Partial Sums

Following the observation of a heavy-tailed distribution in smaller tiles, we quantify the quantization difficulty imposed by outliers through the crest factor ( $\kappa$ ), which is used for block-wise quantization. According to (Chen et al., 2025), the quantization noise is proportional to  $\kappa^2$ . High  $\kappa$  implies that the signal’s dynamic range is dominated by extreme outliers relative to the mean value. Capturing these outliers without clipping results in a coarse allocation of quantization levels and severe degradation in the quantization signal-to-noise ratio (Darvish Rouhani et al., 2023).

However, given the high sparsity inherent in DNN models (e.g., due to ReLU), the standard crest factor becomes unreliable. The abundance of zero values underestimates the global standard deviation, leading to inflated crest factor that fails to represent the true distribution of non-zero elements.

Therefore, we define the “nonzero crest factor ( $\kappa_{nz}$ )” as:

$$\kappa_{nz} = \frac{\max(|X|)}{\sigma_{nz}(X)}, \quad (2)$$

where  $\sigma_{nz}$  denotes the standard deviation calculated over non-zero elements. The  $\kappa_{nz}$  accurately reflects the dispersion of the meaningful signal relative to standard deviation.

Fig. 5 illustrates the box plot of  $\kappa_{nz}$  across different tiling granularities. As the tile size decreases (e.g., from  $256 \times 256$  to  $32 \times 32$ ), the  $\kappa_{nz}$  increases on both ResNet-18 and Llama-3.1. This trend is more prominent in Llama-3.1 since LLMs inherently exhibit more extreme outliers than CNNs. Increased  $\kappa_{nz}$  on smaller tiles can also be interpreted by the well-known central limit theorem. In large tiles, the aggregation of numerous dot products leads to an averaging effect, smoothing the distribution towards a Gaussian shape. The extreme  $\kappa_{nz}$  in small tiles, considering the actual hardware, implies a quantization challenge significantly more severe than quantization on the final output.

### 3.3. Outlier Analysis of Tiled Partial Sums

Despite the high  $\kappa_{nz}$  observed in local tiles, a critical question remains: are these outliers random, or are they deterministic consequences of the input activation? Fig. 6 presents the scatter plots of the maximum absolute activation value versus the maximum absolute psum at two different tile sizes. As the tiling granularity becomes finer, the correlation coefficient ( $r$ ) significantly increases. Notably, in Llama 3.1, the  $32 \times 32$  tiles exhibit a remarkably strong positive correlation of  $r = 0.85$ , compared to  $r = 0.76$  in larger  $128 \times 128$  tiles. Thus, in fine-grained tiles, extreme psums are strongly driven by the extreme input activations. This strong correlation suggests that *input statistics can effectively predict the psum’s upper bound* within the tile. However, due to widely varying weight distributions, the data points exhibit non-negligible residuals rather than perfect linearity. Thus, to handle this complexity, we formulate the scaling problem as a *learnable prediction task based on the input features* (Design Choice #2).

## 4. Scale Calibration Method in PsumQuant

Motivated by the characteristics observed in tiled psums, we present and compare several methods to determine the optimal quantization scale to minimize accuracy loss. A straightforward approach is *dynamic quantization*, which calculates the scale based on real-time statistics, e.g., the maximum value, of the output vector. However, this “output-driven” approach introduces critical hardware and algorithmic inefficiencies in high-throughput NPU architectures.

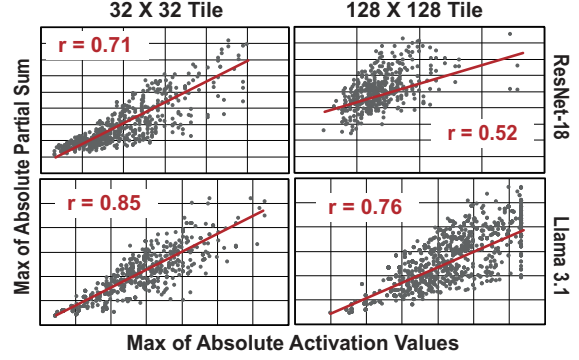


Figure 6. Maximum absolute activation within each tile against the maximum absolute psum for ResNet-18 and Llama-3.1 under two tile granularities.

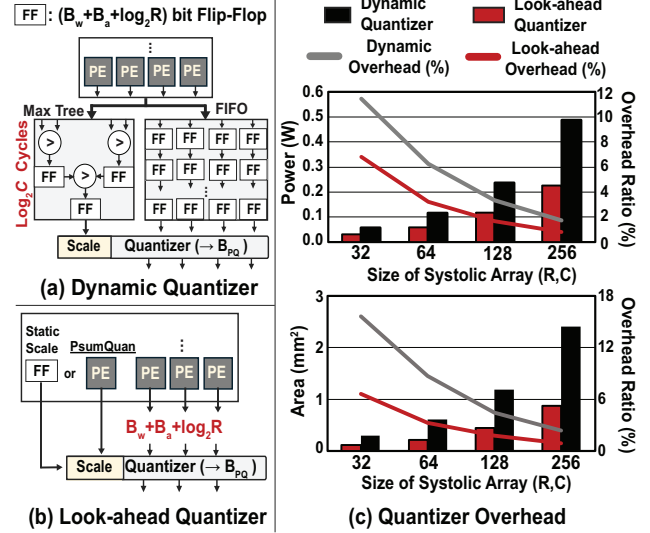


Figure 7. Hardware overhead analysis: max-based dynamic vs. look-ahead quantizers. (a) Max-based dynamic quantizer hardware. (b) Look-ahead architecture. (c) Hardware synthesis results for power and area overheads (65nm CMOS process,  $T_{clk} = 500\text{MHz}$ ).

### 4.1. Limitation of Max-based Dynamic Scaling

In the systolic array, psum vectors of length  $C$  are generated every clock cycle (one vector per cycle after the array is fully filled with streaming data). Implementing real-time dynamic quantization requires finding the maximum value within this vector online. However, the combinational and sequential logic for the max-finder tree scales poly-logarithmically with vector length, i.e.,  $O((\log_2 C)^2)$ . As illustrated in the ‘Dynamic Quantizer’ in Fig. 7(a), pipelining the max-finding path yields a throughput of generating one maximum value (= scale ‘s’) per cycle. However, it introduces a latency of  $L = \lceil \log_2 C \rceil$  cycles, which requires delaying the high-precision psums, i.e.,  $(B_w + B_a + \log_2 R)$ -bit, by  $L$  cycles. This additional buffering consumes significant area and dynamic power, due to flip-flops, nullifying the energy benefits of psum quantization. Consequently, as shown in Fig. 7(c), the area overheads of implementing max-based dynamic quantizer become 4% and 15% at  $128 \times 128$  and  $32 \times 32$

systolic arrays, respectively.

In contrast, the look-ahead quantizer (Fig. 7(b)) eliminates the need for complex max-finder trees and deep FIFO buffers by proactively determining the scale factor at the input feeder. In a systolic array, partial sums are staged in local pipeline registers before being passed to the accumulators or the global buffer. Look-ahead quantizer is strategically placed as combinational logic right before these registers. Since the look-ahead quantizer’s critical path delay is short and easily fits within the timing margin, psums can be quantized to smaller bit-precision before being latched into the registers, requiring no additional clock cycles. According to our hardware synthesis results (Fig. 7(c)) using Synopsys Design Compiler on a 65nm CMOS process, the look-ahead quantizer maintains lower hardware footprint and power consumption than the dynamic quantizer. Notably, as the systolic array scales up to  $128 \times 128$ , the power and area overheads of the look-ahead logic diminish to less than 2% of the total power/area cost of the systolic array.

Furthermore, relying solely on the maximum of absolute values for scaling is not always optimal. While preserving the maximum is essential for outlier-heavy LLMs (Dubey et al., 2024; Zhang et al., 2022), it creates a critical trade-off for models with Gaussian-like distributions, such as CNNs, as scaling based on extremes wastes precision on rare outliers and degrades accuracy. Therefore, achieving both hardware feasibility and algorithmic robustness requires predicting the optimal scale ‘ $s$ ’ by a zero-latency **look-ahead scale predictor** before psum quantization (Fig. 7(b)).

#### 4.2. Tile-wise Learnable Static Scaling

As a look-ahead scaling method, we examine **static scaling**. Note that APSQ (Tan et al., 2025) utilizes static scales through QAT. We assign a unique, learnable scale factor  $S_s$  for every tile. Considering the total dimensions  $M$ ,  $K$ , and  $N$  with respective tile sizes ‘ $m \times R$ ’, ‘ $R \times C$ ’, and ‘ $m \times C$ ’, the total number of scale parameters  $P_{static}$  becomes:

$$P_{static} = \left\lceil \frac{M}{m} \right\rceil \times \left\lceil \frac{K}{R} \right\rceil \times \left\lceil \frac{N}{C} \right\rceil. \quad (3)$$

For each tile, psums denoted by  $x$  are quantized using a tile-wise learnable step size  $S_s$  as follows:

$$\hat{x} = \text{clamp} \left( \left\lfloor \frac{x}{S_s} \right\rfloor, -2^{B_{PQ}-1}, 2^{B_{PQ}-1} - 1 \right) \cdot S_s, \quad (4)$$

where  $\lfloor \cdot \rfloor$  denotes the round-to-nearest operator and the  $B_{PQ}$  represents the bit-width of quantized psums.

To minimize error, we use the **layer-local calibration** (*Design Choice #1*) that optimizes mean squared error (MSE) between the original  $x$  and the quantized output  $\hat{x}$ . Calibration is performed independently for each layer, decoupling

the scale learning from the global parameters. The gradient of the  $S_s$  is derived by differentiating the MSE loss:

$$\frac{\partial \mathcal{L}_{MSE}}{\partial S_s} = (\hat{x} - x) \cdot \frac{\partial \hat{x}}{\partial S_s}, \quad (5)$$

where the term  $(\hat{x} - x)$  represents the quantization error. The partial derivative  $\frac{\partial \hat{x}}{\partial S_s}$  is defined across three distinct regions following the LSQ (Esser et al., 2020) formulation:

$$\frac{\partial \hat{x}}{\partial S_s} \approx \begin{cases} -2^{B_{PQ}-1} & \text{if } \frac{x}{S_s} < -2^{B_{PQ}-1} \\ 2^{B_{PQ}-1} - 1 & \text{if } \frac{x}{S_s} > 2^{B_{PQ}-1} - 1 \\ \left\lfloor \frac{x}{S_s} \right\rfloor - \frac{x}{S_s} & \text{otherwise.} \end{cases} \quad (6)$$

As a result, we can optimize (i) the *clipping error* for values outside the representable range and (ii) the *rounding error* for values within the range. Before training  $S_s$  using calibration data, we initialized  $S_s$  using the theoretical average of the partial sums, derived from the input and weight statistics. Note that given the variable input lengths ( $M$ ) of language models, we applied a shared static scale across the entire sequence (i.e.,  $m = M$ ). Finer-grained static chunking (e.g.,  $m = 1$ ) is structurally ineffective for LLMs because their activation outliers lack positional ( $m$ ) consistency. Under a limited post-training calibration dataset, fine-grained chunking leaves many positional scales untrained, leading to accuracy degradation. Therefore, setting  $m = M$  ensures the most robust and stable static baseline without drastically increasing the required calibration cost.

#### 4.3. PsumQuant: Input-adaptive Dynamic Scaling

Fundamentally motivated by our empirical discovery that input statistics exhibit a strong predictive correlation with the optimal psum quantization scales (*Design Choice #2*), we propose an input-adaptive scaling method (**PsumQuant**). Unlike the static scaling, which relies on pre-determined fixed scales regardless of input variations, our method generates the optimal step size  $S_{PQ}$  on-the-fly by projecting the input activation onto a learnable vector  $\mathbf{v}$ . Specifically, we assign a shared projection vector  $\mathbf{v} \in \mathbb{R}^R$  to each output tile where the total number of output tiles is  $\left\lceil \frac{K}{R} \right\rceil \times \left\lceil \frac{N}{C} \right\rceil$ . For a given input activation vector  $\mathbf{a} \in \mathbb{R}^R$ , the dynamic step size  $S_{PQ}$  is computed by:

$$S_{PQ} = |\mathbf{a} \cdot \mathbf{v}| + b, \quad (7)$$

where the bias  $b$  is initialized to the static scale  $S_s$  and updated using the same formula as in Eq. (6). Importantly, since the vector  $\mathbf{v}$  is shared across the input sequence dimension  $M$ , the total number of scale parameters  $P_{dynamic}$  is decoupled from  $M$ :

$$P_{dynamic} = R \times \left\lceil \frac{K}{R} \right\rceil \times \left\lceil \frac{N}{C} \right\rceil. \quad (8)$$

Structurally, the elements of the projection vector  $\mathbf{v}$  are stored locally within the PEs (Fig. 1) following the WS dataflow *without any hardware modification*. This design allows the predictor to leverage the existing weight reuse mechanism without separate control and memory resources. The computational overhead of this scaling method is negligible. In the ' $R \times C$ ' systolic array generating an psum vector of length  $C$  from an ' $m \times R$ ' input tile, the main matrix multiplication entails ' $R \times C$ ' multiply-accumulate operations (MACs). The proposed PsumQuant involves additional  $R$  MACs simply by utilizing one PE column of the array. Consequently, the computational overhead scales by a factor of  $\frac{1}{C}$  (e.g., less than 1% overhead for  $C = 128$ ), ensuring that the look-ahead scale predictor does not offset the efficiency gains from quantization. Accounting for both the predictor overhead (Fig. 7(c)) and the extra-column scale computation overhead, we confirm that reducing  $B_{PQ}$  from 24-bit to 8-bit yields a **45%** reduction in total average energy on a  $128 \times 128$  weight-stationary systolic array. Similar to the WS array, PsumQuant can be applied to the OS array by dedicating one additional column of the array to compute  $S_{PQ}$ . OS cyclically streams in  $\mathbf{v}$  vectors instead of locally storing them to be multiplied with the incoming input activations. Per  $R$  accumulations, we can invoke local quantization by propagating the locally computed per-row  $S_{PQ}$ . This intermediate quantization with PsumQuant reduces the demand of high bit-width psum registers in the OS dataflow.

To optimize the  $\mathbf{v}$ , we propagate the quantization error back to the predictor parameters. Thus, the gradient of the loss  $\mathcal{L}_{MSE}$  with respect to  $\mathbf{v}$  is formulated as follows:

$$\frac{\partial \mathcal{L}_{MSE}}{\partial \mathbf{v}} = \underbrace{\left( \frac{\partial \mathcal{L}_{MSE}}{\partial \hat{x}} \cdot \frac{\partial \hat{x}}{\partial S_{PQ}} \right)}_{= \partial_{S_s} \mathcal{L}_{MSE} \text{ in Eq. (5)}} \cdot \frac{\partial S_{PQ}}{\partial \mathbf{v}} \cdot \frac{\kappa_{nz}}{\mathbb{E}[\kappa_{nz}]}. \quad (9)$$

The first two terms, representing the derivative of the loss with respect to the scale  $S_{PQ}$ , are identical to the gradient derivation used in the static approach (Eq. (5)). Regarding the projection gradient ( $\frac{\partial S_{PQ}}{\partial \mathbf{v}}$ ), we normalize the input activation  $\mathbf{a}$  by its magnitude to prevent exploding gradients. Without this normalization, outlier-heavy inputs would produce excessively large updates, destabilizing the optimization. Thus, the projection gradient is defined as:

$$\frac{\partial S_{PQ}}{\partial \mathbf{v}} \approx \frac{\mathbf{a}^T}{\|\mathbf{a}\|_1} \cdot \text{sgn}(\mathbf{a} \cdot \mathbf{v}). \quad (10)$$

Last but not least, we incorporate the mean-normalized nonzero crest factor,  $\frac{\kappa_{nz}}{\mathbb{E}[\kappa_{nz}]} \in \mathbb{R}^M$ , as an adaptive weighting term. Since PsumQuant operates on  $M$ -cycles psum vectors, we calculate a unique  $\kappa_{nz}$  for each vector to quantify its outlier severity. High  $\kappa_{nz}$  indicates a distribution dominated by extreme outliers. Multiplying the gradient of each vector

by its corresponding  $\kappa_{nz}$  amplifies the learning for outlier-heavy vectors, forcing the model to achieve a lower MSE.

## 5. Experimental Results

**Experimental Setup** This section presents a comprehensive evaluation of PsumQuant on standard benchmarks for both natural language processing and computer vision. To validate realistic accuracy with both static scaling and PsumQuant methods, we emulate the dataflow of a tiling-based systolic array execution using PyTorch. We employ 8-bit integer quantization for both weights and activations (W8A8) as a default precision using (Xiao et al., 2023; Wei et al., 2022). We utilize a minimal subset of unlabelled data, strictly adhering to standard PTQ practices: 100 randomly sampled images for ImageNet and 16 sequences for LLMs. Learnable scale parameters ( $S_s, \mathbf{v}$ ) are calibrated for 100 epochs by a single NVIDIA RTX 6000 GPU (90GB), yielding stable convergence.

The calibration overhead scales inversely with the NPU array size. Because smaller arrays require more tiles to compute the same matrix multiplication, the number of scale calibrations increases. However, even in the most compute-intensive array size, calibrating the projection vector over 100 epochs takes under 1 hour (33.6 sec/epoch) with the peak GPU memory strictly bounded to 41 GB. In contrast, QAT with the same number of calibration samples incurred a  $3.5\times$  latency overhead compared to our lightweight PTQ approach. While PsumQuant effectively calibrates using only 16 sequences, standard LLM QAT requires tens of thousands of token sequences. This demonstrates that our PTQ-based calibration is highly resource-efficient and can be fully executed on a single GPU.

Table 1. Calibration overhead of PsumQuant for the Llama-3.1 8B model on a single RTX 6000 GPU.

| NPU Array Size  | 256 × 256 | 128 × 128 | 64 × 64 |
|-----------------|-----------|-----------|---------|
| Time (min)      | 18.92     | 30.58     | 55.97   |
| Peak GPU Memory | 21 GB     | 33 GB     | 41 GB   |

**Buffer-level Accumulation Bit-width** To handle the magnitude increase when accumulating  $\frac{K}{R}$  psum tiles at a global buffer, we set a wider bit-width for accumulation ( $B_{acc}$ ) than the psum bit-width ( $B_{PQ}$ ). Crucially, since each psum vector possesses a distinct scale, we align them to the peak scale factor observed across the accumulation using shift-add operations. Thus, we set

$$B_{acc} = B_{PQ} + \left\lceil \log_2 \left( \frac{K}{R} \right) \right\rceil,$$

which provides the necessary arithmetic margin to strictly prevent overflow under the worst-case scenario, when all

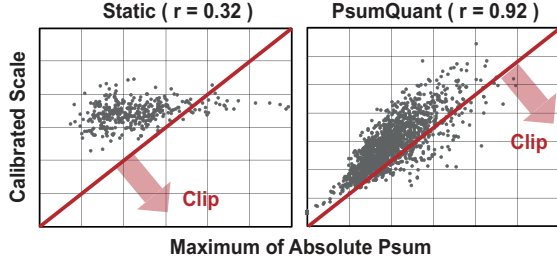


Figure 8. Correlation between the calibrated scale and the maximum of absolute psums tested on ResNet-18. (Left) Static scaling exhibits weak correlation ( $r = 0.32$ ) due to fixed parameters. (Right) PsumQuant demonstrates strong linearity ( $r = 0.92$ ), effectively tracking the maximum magnitude of partial sums.

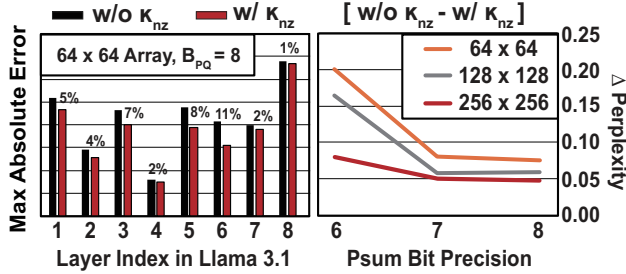


Figure 9. Effectiveness of  $\kappa_{nz}$ -based gradient modulation. (Left) Reduction in layer-wise peak errors (annotated in %). (Right) Resulting perplexity gains ( $\Delta$ PPL).

psums have the same sign. By minimizing the quantization bit-width to  $B_{PQ}$ , we can reduce the required buffer size, leading to a more power-efficient hardware implementation.

**Comparison with Static Scaling** A comparison between scaling methods reveals the inefficiency of the static scaling (Fig. 8) which is commonly adopted in prior works. Relying on a fixed scaling factor determined by global statistics, static quantization consistently overestimates the required range for actual psums, resulting in scales far exceeding the actual psum magnitudes. This unnecessary headroom results in wasted quantization resolution. Despite this conservative approach, it fails to capture extreme dynamic peaks, leading to clipping errors at outliers leading to low correlation ( $r = 0.32$ ). In contrast, the scale extracted by PsumQuant aligns well with the psum magnitude ( $r = 0.92$ ), effectively eliminating both the resolution waste for small inputs and the clipping noise for outliers.

**$\kappa_{nz}$ -based Calibration** Fig. 9(left) illustrates the effectiveness of our  $\kappa_{nz}$ -based gradient modulation in mitigating quantization noise. Despite varying error magnitudes across layers, incorporating  $\kappa_{nz}$  consistently reduces the maximum absolute error relative to the baseline. This layer-wise analysis confirms that our method successfully suppresses global peak errors, contributing to the improved robustness observed in the final perplexity results. The quantitative impact of this strategy is evident in the perplexity analy-

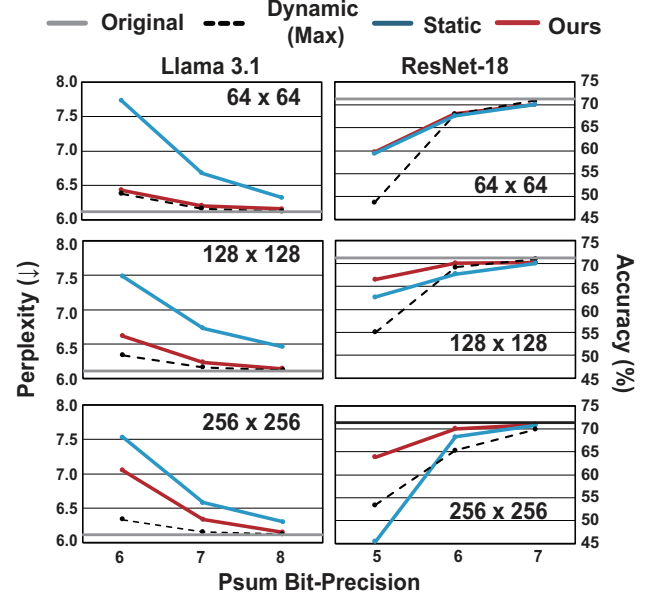


Figure 10. Performance comparison on Llama 3.1 (Perplexity  $\downarrow$ ) and ResNet-18 (Accuracy  $\uparrow$ ) across various tile sizes. Our method (red) closely approaches the dynamic (max) baseline (black dashed) in Llama 3.1 and surpasses it in ResNet-18 at low bit-widths, outperforming the static baseline (blue).

sis Fig. 9(right). While the performance gap is marginal at 8-bit, it widens dramatically as the precision drops to an aggressive 6-bit regime. This sharp increase indicates that our method acts as a vital safety net, preventing the performance collapse observed in the baseline under tight constraints. Notably, this stabilization is most pronounced at smaller tile sizes, supporting our analysis in Section 3, which shows that outliers exert a significantly larger impact on quantization quality as tile size decreases.

**Robustness on Tiling Sizes** Fig. 10 demonstrates the robustness of our method against static and dynamic (max-based) quantization baselines across various tiling configurations. As percentile-based psum clipping caused severe accuracy degradation, we therefore omit it from the comparisons. For Llama 3.1 on WikiText-2 (Merity et al., 2017), the static baseline suffers significant degradation at all hardware configurations at 6-bit. In contrast, PsumQuant effectively mitigates quantization noise, recovering PPL to the dynamic method at 7/8-bit. The dynamic quantization seems promising on LLMs where significant outliers exist. However, for models with Gaussian-like distributions such as ResNet-18, PsumQuant shows more stable accuracy than the dynamic method at low  $B_{PQ}$  with lower hardware cost (Fig. 7). Also, PsumQuant consistently shows better stability, matching or exceeding the accuracy of static quantization at all array sizes. Overall, with PsumQuant, we can safely quantize psums for LLMs at 8-bit and for vision models at 7-bit. We further confirmed that PsumQuant remains robust on practical small arrays: at  $B_{PQ}$  Llama-3.1 8B shows only +0.07

Table 2. Zero-shot perplexity/accuracy evaluation with a  $128 \times 128$  systolic array. We apply SmoothQuant to obtain a W8A8 quantization across all methods. We vary the psum handling: (i) INT32 psum (baseline), (ii) static psum quantization, and (iii) PsumQuant.

| Model           | Method           | Perplexity ↓ | PIQA ↑      | HellaSwag↑  | WinoGrande↑ | ARC-c↑      | OpenBookQA↑ | Avg↑        |
|-----------------|------------------|--------------|-------------|-------------|-------------|-------------|-------------|-------------|
| Llama 3.1 (8B)  | INT32 Psum       | 6.10         | 80.6        | 79.2        | 74.1        | 55.0        | 45.8        | 66.9        |
|                 | Static           | 6.46         | 80.1        | 79.1        | 73.5        | 53.5        | 44.5        | 66.1        |
|                 | <b>PsumQuant</b> | <b>6.14</b>  | <b>80.5</b> | <b>79.0</b> | <b>73.4</b> | <b>54.0</b> | <b>45.0</b> | <b>66.4</b> |
| Phi-3 (3.8B)    | INT32 Psum       | 6.26         | 80.8        | 77.0        | 73.1        | 58.2        | 46.6        | 67.1        |
|                 | Static           | 6.54         | 79.5        | 76.8        | 71.1        | 57.9        | 46.4        | 66.3        |
|                 | <b>PsumQuant</b> | <b>6.27</b>  | <b>80.5</b> | <b>77.0</b> | <b>71.2</b> | <b>57.7</b> | <b>46.6</b> | <b>66.6</b> |
| Qwen-2.5 (1.5B) | INT32 Psum       | 8.84         | 75.0        | 67.7        | 64.6        | 43.3        | 40.6        | 58.2        |
|                 | Static           | 9.17         | 75.2        | 67.0        | 62.7        | 43.9        | 39.0        | 57.6        |
|                 | <b>PsumQuant</b> | <b>8.87</b>  | <b>75.1</b> | <b>67.3</b> | <b>63.1</b> | <b>43.3</b> | <b>41.4</b> | <b>58.0</b> |

and +0.06 PPL degradation on  $16 \times 16$  and  $32 \times 32$  arrays, respectively.

**Benchmark Evaluation** We conducted zero-shot evaluations using the LM Evaluation Harness framework (Gao et al., 2021) on PIQA (Bisk et al., 2020), HellaSwag (Zellers et al., 2019), WinoGrande (Sakaguchi et al., 2021), ARC-Challenge (Clark et al., 2018), and OpenBookQA (Mihaylov et al., 2018). As shown in Table 2, our method consistently outperforms the static baseline and achieves near-lossless performance, comparable to INT32-psum baseline (within 0.5% accuracy loss). This proves that our scale prediction scales effectively from standard 8B models (Llama 3.1) down to highly compact 1B-3B models (Qwen 2.5 (Yang et al., 2024) and Phi-3 (Abdin et al., 2024)). This generalizability extends to computer vision (Table 3), where our approach secures significant improvements over the static method for both CNNs and a vision transformer. By effectively recovering from accuracy drops caused by static clipping, our method demonstrates broad applicability across various neural architectures and tiling strategies.

Table 3. ImageNet Top-1 accuracy (%) under  $B_{PQ} = 6$  on a  $128 \times 128$  array compared to baseline with INT32 and Static

| Model     | INT32 | Static | PsumQuant    | Improv. |
|-----------|-------|--------|--------------|---------|
| ResNet-18 | 71.12 | 67.66  | <b>70.04</b> | +2.38   |
| ResNet-34 | 76.18 | 73.97  | <b>74.57</b> | +0.60   |
| Swin-T    | 81.24 | 78.78  | <b>81.02</b> | +2.24   |

## 6. Conclusion

To enable energy-efficient inference on resource-constrained NPUs, this work proposes a lightweight, look-ahead scale predictor that enables input-aware tile-level scaling via a learnable vector  $\mathbf{v}$ . Instead of relying on computationally expensive global retraining, we propose layer-wise scale calibration that minimizes the MSE for each layer independently, enabling highly cost-effective deployment. Our  $\kappa_{nz}$ -

aware optimization preserves model performance on outliers, securing competitive results even in low-bit hardware environments. Evaluations on LLMs and CNNs demonstrate that our PsumQuant significantly outperforms static baselines. With minimal accuracy loss, our approach reduces inference energy by 45% on a  $128 \times 128$  systolic array through robust 8-bit partial-sum quantization, paving the way for scalable, energy-efficient on-device AI deployment. Although we focus on integer-based inference, the psum traffic bottleneck remains relevant for microscaling formats such as MXFP4/8, where block-shared exponents reduce operand precision but higher-precision accumulation is still required. Extending PsumQuant to these formats is therefore an important direction for future work.

## Acknowledgements

This work was partially supported by the Institute of Information & Communications Technology Planning & Evaluation (IITP) funded by the Korea government (MSIT) under Grant RS-2023-00229849 and Grant RS 2025-25442405; in part by the National Research Foundation of Korea (NRF) funded by the Ministry of Science and ICT under Grant NRF 2023R1A2C2006290.

## Impact Statement

With the explosive growth of AI-powered edge and mobile devices, ensuring their energy consumption remains sustainable has become a critical environmental concern. Our work directly addresses this challenge by introducing a hardware-efficient partial sum quantization mechanism. By effectively reducing the energy-intensive data communication between the on-chip buffer and the computing core, our proposed method significantly lowers the dynamic power required for inference. This advancement serves as a key enabler for sustainable on-device AI, minimizing the total energy cost per token generation across billions of devices without compromising model quality.

## References

- Abdin, M., Jacobs, S. A., Awan, A. A., Aneja, J., Ahmed, A. A., Barone, R., Behl, R., Benny, B., Bhowmick, A., Cheng, W.-C., et al. Phi-3 technical report: A highly capable language model locally on your phone. *arXiv preprint arXiv:2404.14219*, 2024.
- Ashkboos, S., Mohtashami, A., Croci, M. L., Li, B., Cameron, P., Jaggi, M., Alistarh, D., Hoefler, T., and Hensman, J. Quarot: Outlier-free 4-bit inference in rotated llms. *Proceedings of Advances in Neural Information Processing Systems (NeurIPS)*, 37:100213–100240, 2024.
- Bai, J., Sun, S., Zhao, W., and Kang, W. Cimq: A hardware-efficient quantization framework for computing-in-memory-based neural network accelerators. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 43(1):189–202, 2023.
- Bisk, Y., Zellers, R., Le Bras, R., Gao, J., and Choi, Y. Piqa: Reasoning about physical commonsense. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, volume 34, pp. 7432–7439, 2020.
- Cerebras Systems. Whitepapers. Cerebras AI Whitepapers Webpage, 2026. URL <https://www.cerebras.ai/whitepapers>. Accessed: 2026-01-23.
- Chen, M., Wu, M., Jin, H., Yuan, Z., Liu, J., Zhang, C., Li, Y., Huang, J., Ma, J., Xue, Z., et al. INT vs FP: A comprehensive study of fine-grained low-bit quantization formats. *arXiv:2510.25602*, 2025.
- Choi, J., Wang, Z., Venkataramani, S., Chuang, P. I.-J., Srinivasan, V., and Gopalakrishnan, K. Pact: Parameterized clipping activation for quantized neural networks. *arXiv:1805.06085*, 2018.
- Clark, P., Cowhey, I., Etzioni, O., Khot, T., Sabharwal, A., Schoenick, C., and Tafjord, O. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.
- Colbert, I., Pappalardo, A., and Petri-Koenig, J. A2Q: Accumulator-aware quantization with guaranteed overflow avoidance. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 16989–16998, 2023.
- Courbariaux, M., Hubara, I., Soudry, D., El-Yaniv, R., and Bengio, Y. Binarized neural networks: Training deep neural networks with weights and activations constrained to +1 or -1. *arXiv:1602.02830*, 2016.
- Darvish Rouhani, B., Zhao, R., Elango, V., Shafipour, R., Hall, M., Mesmakhosroshahi, M., More, A., Melnick, L., Golub, M., Varatkar, G., et al. With shared microexponents, a little shifting goes a long way. In *Proceedings of the International Symposium on Computer Architecture (ISCA)*, pp. 1–13, 2023.
- Dettmers, T., Pagnoni, A., Holtzman, A., and Zettlemoyer, L. QLoRA: Efficient finetuning of quantized LLMs. *Advances in Neural Information Processing Systems*, 36:10088–10115, 2023.
- Dong, Z., Yao, Z., Gholami, A. R., Mahoney, M. W., and Keutzer, K. HAWQ: Hessian aware quantization of neural networks with mixed-precision. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 293–302, 2019.
- Du, Z., Fasthuber, R., Chen, T., Ienne, P., Li, L., Luo, T., Feng, X., Chen, Y., and Temam, O. ShiDianNao: Shifting vision processing closer to the sensor. In *Proceedings of the International Symposium on Computer Architecture (ISCA)*, pp. 92–104, 2015.
- Dubey, A., Jauhri, A., Pandey, A., Keneally, A., Zheng, C.-Y., Grattafiori, N., et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Esser, S. K., McKinstry, J. L., Bablani, D., Appuswamy, R., and Modha, D. S. Learned step size quantization. In *Proceedings of International Conference on Learning Representations*, 2020.
- Gao, L., Tow, J., Abbasi, Biderman, S., Black, S., Golding, A., Hoppe, T., Foster, C., Phang, J., He, H., Thite, A., Nabeshima, N., Presser, S., and Leahy, C. A framework for few-shot language model evaluation. <https://zenodo.org/record/5371628>, 2021. v0.0.1.
- Google Cloud. TPU architecture. Google Cloud Documentation, 2026. URL <https://cloud.google.com/tpu/docs/system-architecture-tpu-vm>. Last updated 2026-01-22 UTC. Accessed 2026-01-23.
- Groq. What is a language processing unit? Groq Whitepapers, 2026. URL <https://groq.com/papers>. Accessed: 2026-01-23.
- Han, S., Mao, H., and Dally, W. J. Deep compression: Compressing deep neural network with pruning, trained quantization and Huffman coding. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2016.
- Hwang, S., Lee, S., Park, D., Lee, D., and Kung, J. SpikedAttention: Training-free and fully spike-driven transformer-to-SNN conversion with winner-oriented spike shift for softmax operation. *Proceedings of Advances in Neural Information Processing Systems (NeurIPS)*, 37:67422–67445, 2024.

- Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., and Kalenichenko, D. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2704–2713, 2018.
- Jouppi, N. P., Young, C., Patil, N., Patterson, D., Agrawal, G., Bajwa, R., Bates, S., Bhatia, S., Boden, N., Borchers, A., et al. In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the International Symposium on Computer Architecture (ISCA)*, pp. 1–12, 2017.
- Kim, J., Jeon, K. E., Kim, Y., and Ko, J. H. Column-wise quantization of weights and partial sums for accurate and efficient compute-in-memory accelerators. In *2025 Design, Automation & Test in Europe Conference (DATE)*, pp. 1–7. IEEE, 2025.
- Kim, Y., Kim, H., and Kim, J.-J. Extreme partial-sum quantization for analog computing-in-memory neural network accelerators. *ACM Journal on Emerging Technologies in Computing Systems (JETC)*, 18(4):1–19, 2022.
- Li, Y., Gong, R., Tan, X., Yang, Y., Hu, P., Zhang, Q., Yu, F., Wang, W., and Gu, S. Brecq: Pushing the limit of post-training quantization by block reconstruction. In *Proceedings of International Conference on Learning Representations (ICLR)*, 2021.
- Merity, S., Xiong, C., Bradbury, J., and Socher, R. Pointer sentinel mixture models. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2017.
- Mihaylov, T., Clark, P., Khot, T., and Sabharwal, A. Can a suit of armor conduct electricity? a new dataset for open book question answering. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 2381–2391, 2018.
- Nagel, M., Amjad, R. A., Van Baalen, M., Louizos, C., and Blankevoort, T. Up or down? adaptive rounding for post-training quantization. In *Proceedings of the International Conference on Machine Learning (ICML)*, pp. 7197–7206, 2020.
- Parashar, A., Raina, P., Shao, Y. S., Chen, Y.-H., Ying, V. A., Mukkara, A., Venkatesan, R., Khailany, B., Keckler, S. W., and Emer, J. Timeloop: A systematic approach to DNN accelerator evaluation. In *Proceedings of the IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pp. 304–315, 2019.
- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., and Sutskever, I. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.
- Rastegari, M., Ordonez, V., Redmon, J., and Farhadi, A. XNOR-Net: Imagenet classification using binary convolutional neural networks. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 525–542, 2016.
- Sakaguchi, K., Bras, R. L., Bhagavatula, C., and Choi, Y. Winogrande: an adversarial winograd schema challenge at scale. *Commun. ACM*, 64(9):99–106, August 2021. ISSN 0001-0782.
- Saxena, U., Chakraborty, I., and Roy, K. Towards adless compute-in-memory accelerators for energy efficient deep learning. In *2022 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pp. 624–627. IEEE, 2022.
- Tan, Y., Dong, P., Wu, Y., Liu, Y., Liu, X., Luo, P., Liu, S.-Y., Huang, X., Zhang, D., Liang, L., et al. Apsq: Additive partial sum quantization with algorithm-hardware co-design. In *2025 62nd ACM/IEEE Design Automation Conference (DAC)*, pp. 1–7. IEEE, 2025.
- Tavanaei, A., Ghodrati, M., Kheradpisheh, S. R., Masque-lier, T., and Maida, A. Deep learning in spiking neural networks. *Neural Networks*, 111:47–63, 2019.
- Wei, X., Gong, R., Li, Y., Liu, X., and Yu, F. Qdrop: Randomly dropping quantization for extremely low-bit post-training quantization. In *Proceedings of International Conference on Learning Representations (ICLR)*, 2022.
- Xiao, G., Lin, J., Seznec, M., Wu, H., Demouth, J., and Han, S. SmoothQuant: Accurate and efficient post-training quantization for large language models. In *Proceedings of the International Conference on Machine Learning (ICML)*, pp. 38087–38099, 2023.
- Yang, A., Yang, B., Hui, B., Pan, B., Tian, B., Zhou, C., Cheng, C., Li, D., Tu, C., Han, C., et al. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*, 2024.
- Zellers, R., Holtzman, A., Bisk, Y., Farhadi, A., and Choi, Y. Hellaswag: Can a machine really finish your sentence? In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 4791–4800, 2019.
- Zhang, S., Roller, S., Goyal, N., Artetxe, M., Chen, M., Chen, S., Dewan, C., Diab, M., Li, X., Lin, X. V., et al. OPT: Open pre-trained transformer language models. *arXiv:2205.01068*, 2022.

## A. Impact of LLM Pre-Conditioning

The proposed PsumQuant is inherently compatible with various input and weight quantization schemes, including naive (min-max) scaling and outlier smoothing. To demonstrate that PsumQuant’s effectiveness is not strictly reliant on activation pre-conditioning, we evaluated it under a naive W8A8 quantization baseline (without SmoothQuant) across various array sizes. As shown in Table 4, the perplexity degradation caused by 8-bit PsumQuant compared to the unquantized psum (INT32) baseline is practically identical in both scenarios (e.g., a marginal PPL degradation of +0.04 to +0.05, regardless of the presence of SmoothQuant).

Table 4. Llama-3.1 8B perplexity with 8-bit PsumQuant at various array sizes.

| Configuration   | INT32 Psum | 256 × 256    | 128 × 128    | 64 × 64      |
|-----------------|------------|--------------|--------------|--------------|
| w/o SmoothQuant | 6.12       | 6.17 (+0.05) | 6.16 (+0.04) | 6.16 (+0.04) |
| w/ SmoothQuant  | 6.10       | 6.15 (+0.05) | 6.14 (+0.04) | 6.14 (+0.04) |

Without SmoothQuant, the massive outliers typical of LLM activations remain unmitigated. Consequently, during accumulation, these raw activation outliers directly induce more extreme, heavy-tailed outliers in the psum distribution than the smoothed baseline. Despite these harsher conditions, PsumQuant maintains its accuracy thanks to its core algorithmic design: **input-aware dynamic scaling**. Because our look-ahead predictor observes the incoming input activations on-the-fly, it immediately detects when an outlier is being generated within the array. It dynamically scales up the step size ( $S_{PQ}$ ), effectively handling extreme psums without severe clipping errors. In short, the quality of PsumQuant does not depend on the underlying pre-conditioning scheme; its input-aware mechanism dynamically adapts to the severity of the given outliers.

## B. Impact of Calibration Dataset Size

Table 5. Perplexity of Llama-3.1 8B with respect to the number of calibration samples ( $B_{PQ} = 7$ ).

| # Calibration Samples | 4    | 8    | 16   | 32   | 64   |
|-----------------------|------|------|------|------|------|
| Perplexity (PPL)      | 6.26 | 6.24 | 6.23 | 6.23 | 6.22 |

To analyze the impact of the calibration dataset size, we evaluated PsumQuant using 4 to 64 sequences. The perplexity (PPL) of the Llama-3.1 8B model on WikiText-2 was measured with 7-bit psums on a  $128 \times 128$  array configuration. As demonstrated in Table 5, the 16-sequence baseline does not suffer from overfitting. Doubling the dataset to 32 sequences yields no additional performance gain (6.23), and quadrupling it to 64 sequences results in a negligible 0.01 improvement (6.22). Therefore, we deliberately selected 16 sequences to minimize calibration overhead and enable

rapid deployment.

Furthermore, unlike the pre-trained LLM weights, the projection vectors ( $v$ ) in PsumQuant are completely uninitialized parameters. Therefore, they require a sufficient number of gradient descent steps to converge properly. Running 100 epochs for an  $R$ -length vector  $v$  is a necessary step to stabilize the training of the dynamic scale factors without significantly increasing the computational burden.

## C. Comprehensive Energy Analysis and Hardware Robustness

This section provides an expanded analysis of the energy savings achieved by PsumQuant, detailing the consumption across the memory hierarchy and demonstrating the method’s robustness across various array sizes.

### C.1. Robustness Across Array Sizes

Table 6. Total energy consumption (mJ) with respect to array sizes: 8-bit vs. 32-bit partial sums.

| Array Size       | 8-bit Psum | 32-bit Psum | Reduction (%) |
|------------------|------------|-------------|---------------|
| $32 \times 32$   | 282.42     | 702.61      | 59.80         |
| $64 \times 64$   | 223.85     | 636.89      | 64.85         |
| $128 \times 128$ | 197.06     | 595.75      | 66.92         |
| $256 \times 256$ | 184.02     | 554.79      | 66.83         |

Table 6 illustrates the total energy consumption for various array sizes (from  $32 \times 32$  to  $256 \times 256$ ) under a Weight-Stationary (WS) dataflow. The total energy encompasses the operations across the entire memory hierarchy—including the local scratchpad (SPAD), global buffer (GLB), and DRAM—during input read, weight pre-load, and output update phases. For all array sizes, the energy consumed by inputs and weights remains constant since they are uniformly quantized to 8-bit (W8A8). For the unquantized 32-bit psum baselines, output-related data transfer energy strictly dominates the total energy consumption, regardless of the array dimension (e.g., accounting for 596.8 mJ out of 702.6 mJ, or 84.9%, in the  $32 \times 32$  array). By compressing the psum data to 8-bit, PsumQuant universally slashes this dominant output energy, driving a massive reduction in total energy. While total energy inherently increases as the array size shrinks due to more frequent K-dimension tiling, the output energy remains the prominent bottleneck in every configuration. This confirms that our energy reduction is universal and not limited to specific hardware dimensions.

### C.2. Detailed Hardware Energy Breakdown

Table 7 provides a detailed component-wise energy breakdown for 8-bit versus 24-bit psums. In our Timeloop-Accelerergy simulation setup based on the TPU-like  $128 \times 128$

systolic array, input activations and weights are strictly fixed at 8 bits (W8A8) across all configurations. Consequently, the energy consumed by the multipliers (MAC), as well as the input and weight memory hierarchies (SRAM buffers and routing), remains completely constant.

Table 7. Detailed hardware energy breakdown (mJ) across the memory hierarchy ( $128 \times 128$  array).

| Hardware Component    | 8-bit Psum    | 24-bit Psum   |
|-----------------------|---------------|---------------|
| MAC                   | 134.98        | 134.98        |
| SPAD (Input + Weight) | 60.06         | 60.06         |
| SPAD (Output)         | 125.69        | 355.17        |
| GLB (Input + Weight)  | 16.99         | 16.99         |
| GLB (Output)          | 13.42         | 64.45         |
| DRAM (Input + Weight) | 26.40         | 26.40         |
| DRAM (Output)         | 10.09         | 10.19         |
| <b>Total Energy</b>   | <b>387.63</b> | <b>668.24</b> |

The primary variable in our evaluation is the psum bit-width, which dictates the required bit-width for the global buffer and accumulators. As demonstrated in Table 7, when the output precision increases from 8-bit to 24-bit, the local output scratchpad (SPAD) energy increases by approximately 2.8 times (from 125.69 mJ to 355.17 mJ), and the output global buffer (GLB) energy dramatically increases by 4.8 times (from 13.42 mJ to 64.45 mJ). It demonstrates that while the multiplication and input/weight routing energies remain static, psum storage and data movement constitute the overwhelming energy bottlenecks. By dynamically compressing these psums, PsumQuant effectively reduces this bottleneck.